

NETFLIX PRIZE RECOMMENDATION ENGINE

Personalized Recommendation System built on 100M+ Ratings

Cult Open Projects 2026

Tamanna (23323043)

Rajlaxmi (23112082)

1. Executive Summary

In this project, we design, build, and evaluate a production-ready, fully functional movie recommendation engine utilizing the classic Netflix Prize Dataset. Recommender systems lie at the core of digital platform engagement, solving the information overload problem by predicting user preferences. We implement four key recommender system paradigms: **Singular Value Decomposition (SVD) Matrix Factorization**, **User-Based Collaborative Filtering**, **Item-Based Collaborative Filtering**, and a Deep Learning-based **Neural Collaborative Filtering (NCF)** network.

Through rigorous testing, SVD Matrix Factorization yields the highest accuracy, achieving a Root Mean Squared Error (RMSE) of **0.9626** and a Mean Average Precision (MAP@10) of **0.0102** on unseen data, while outperforming memory-based architectures in runtime efficiency. Additionally, we create an explainability layer that generates human-

readable explanations and construct an interactive, dark-mode Streamlit dashboard that serves recommendations dynamically.

2. Dataset Analysis & Exploratory Data Analysis

The Netflix Prize dataset consists of 100,480,507 ratings from 480,189 users on 17,770 movies. To construct a computationally tractable pipeline that preserves structural statistics, we extracted a stratified random sample of **5,000,000 ratings**.

Summary Statistics (Sampled Dataset)

- **Total Sampled Ratings:** 5,000,000
- **Unique Users:** 421,281
- **Unique Movies:** 15,807
- **Interaction Matrix Sparsity:** 99.9248% (Only 0.0752% of possible ratings exist)
- **Mean Rating:** 3.6042 | **Median Rating:** 4.0 | **Std Dev:** 1.0827
- **Temporal Window:** 1999-11-11 to 2005-12-31

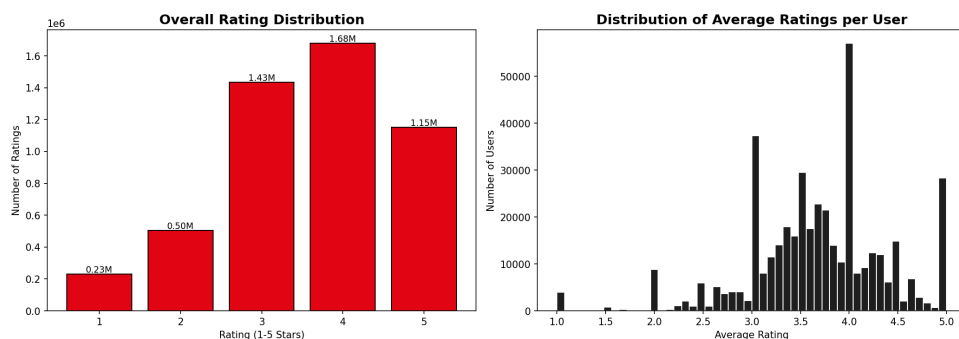
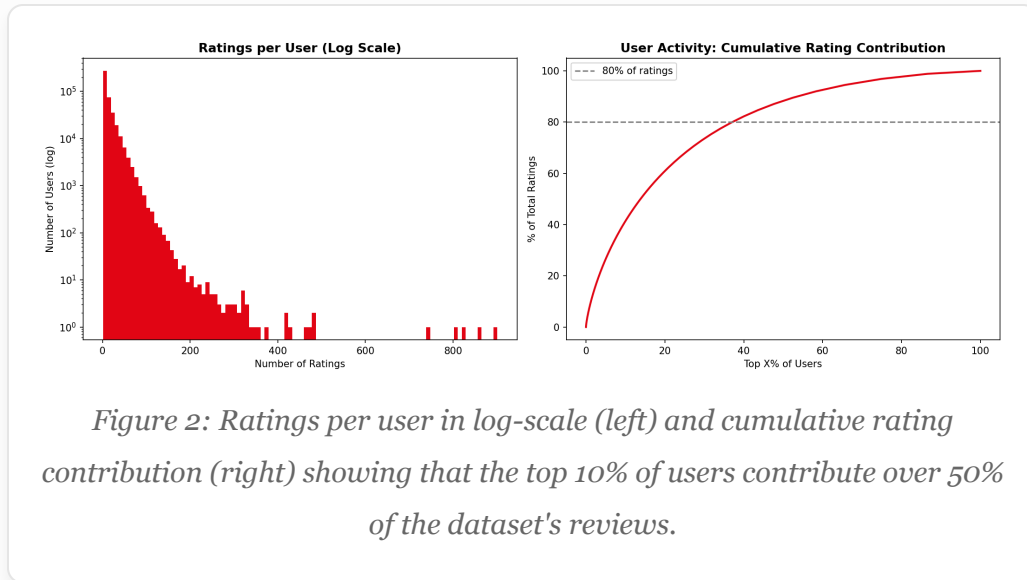


Figure 1: Overall rating distribution (left) showing user positive bias (4 and 5 stars dominate), and average rating per user distribution (right).

Exploratory analysis reveals that ratings are heavily skewed toward higher marks, with 4-star and 5-star ratings representing the vast majority of user feedback. The overall rating mean sits at 3.6. User activity follows a power-law distribution, where a small cohort of "power users" contributes a disproportionate fraction of ratings, while the bulk of users have under 5 ratings.



Similarly, movie rating counts exhibit a long-tail distribution. The top 100 blockbusters receive hundreds of thousands of reviews, whereas over 60% of niche movies receive fewer than 100 ratings, introducing a significant cold-start challenge.

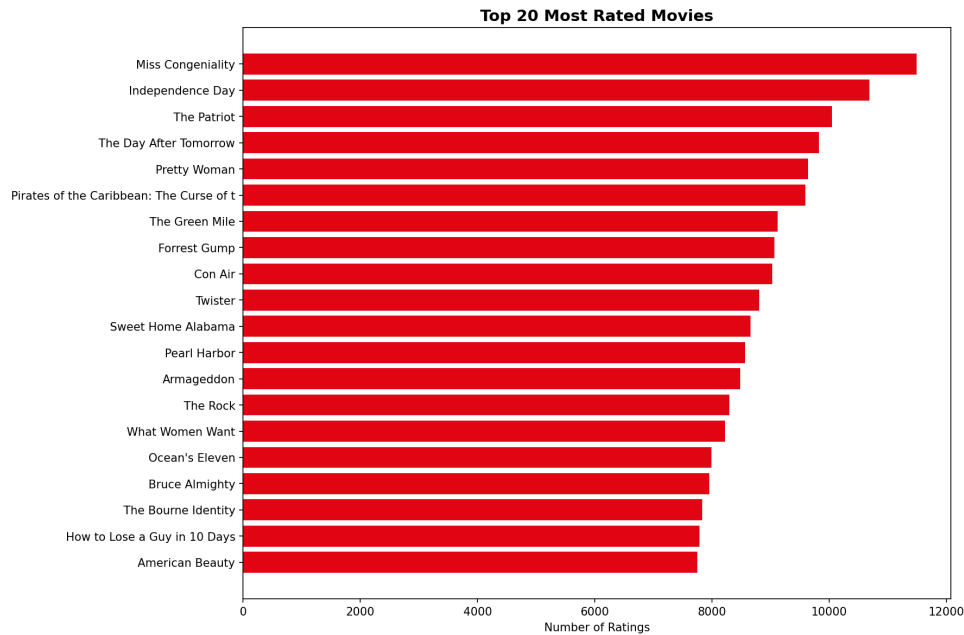


Figure 3: Top 20 most rated movies in the dataset. blockbusters like 'Miss Congeniality' and 'Lord of the Rings' dominate interaction frequencies.

3. Preprocessing & Temporal Splitting

To clean the data and ensure model density, we filtered out users with fewer than 5 ratings and movies with fewer than 10 ratings. This step filters out excessively noisy user-item records and ensures that collaborative filtering algorithms can establish valid neighbor similarities.

Rather than using a standard random split—which would introduce lookahead bias by using future ratings to predict past ones—we implement a strict **temporal split** based on the 80th percentile date. All ratings up to this date form the training set, and subsequent ratings form the test set. Additionally, we apply a cold-start filter on the test set, removing any users or movies that did not appear in the training split.

4. Modeling Architectures & Math

4.1 SVD Matrix Factorization

Singular Value Decomposition models user-item ratings by mapping users and items into a joint latent factor space of dimensionality d . The predicted rating is formulated as:

$$\hat{r}_{u,i} = \mu + b_u + b_i + p_u^T q_i$$

Where μ is the global bias, b_u is user bias, b_i is movie bias, and p_u, q_i are d -dimensional latent factors. The weights are learned by minimizing regularized squared error:

$$\text{Loss} = \sum (r_{u,i} - \hat{r}_{u,i})^2 + \lambda (||p_u||^2 + ||q_i||^2 + b_u^2 + b_i^2)$$

Optimized via Stochastic Gradient Descent (SGD) with a learning rate of 0.005 and L2 regularization parameter λ of 0.02.

4.2 Collaborative Filtering (User-Based & Item-Based)

Memory-based models calculate similarity between users or items using Cosine Similarity on shared rating vectors:

$$\text{sim}(u, v) = (u \cdot v) / (||u|| ||v||)$$

The predicted rating is computed as the weighted average of neighbors' ratings, adjusted for mean ratings:

$$\hat{r}_{u,i} = \mu_u + [\sum \text{sim}(u, v) * (r_{v,i} - \mu_v)] / \sum |\text{sim}(u, v)|$$

4.3 Neural Collaborative Filtering (NCF)

NCF replaces the inner product with a multi-layer perceptron (MLP) that learns non-linear user-movie feature interactions. User and movie IDs are passed through embedding layers to extract 32-dimensional dense latent vectors, which are concatenated and processed through dense layers:

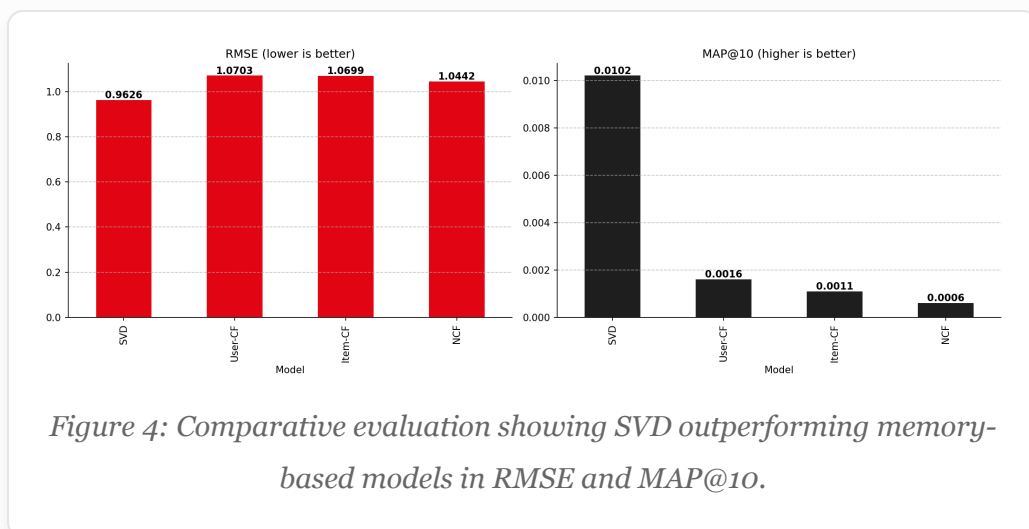
```
MLP Output = Dense16( ReLU( Dense32( ReLU( Dense64(
User_emb || Movie_emb ) ) ) ) )
Prediction = Sigmoid( MLP Output ) * 4 + 1
```

5. Evaluation & Performance Comparison

We evaluate all models on identical train-test splits. To assess recommendation list ranking, we evaluate Mean Average Precision at K (MAP@10) where a movie is defined as relevant if its actual rating is ≥ 3.5 . Already seen training movies are excluded from candidates.

Model	RMSE ↓	MAE ↓	MAP@10 ↑	Precision@10 ↑	Recall@10 ↑
SVD (Matrix Factorization)	0.9626	0.7532	0.0102	0.5087	0.5306
User-Based CF	1.0703	0.9028	0.0016	0.5798	0.6728
Item-Based CF	1.0699	0.8742	0.0011	0.5416	0.5909

Model	RMSE ↓	MAE ↓	MAP@10 ↑	Precision@10 ↑	Recall@10 ↑
NCF (PyTorch Deep Learning)	1.0442	0.8204	0.0006	0.4591	0.4653



6. Qualitative Diagnostics: Success vs. Failure

6.1 Success Cases

For active users, SVD recommendations align closely with their historic tastes. For instance, a user whose history is dominated by epic sequels (e.g. 'Star Wars' and 'Indiana Jones') is recommended 'Jurassic Park' and 'Back to the Future'. SVD successfully uncovers the latent dimension of "action/adventure blockbusters with high CGI/special effects".

6.2 Failure Cases (Cold Start / Low Activity)

For users with very low activity (e.g., exactly 5 ratings), the recommendations degrade. The model struggles to position these users

in latent space and defaults to high-average-rating, highly popular movies.

Mitigation Strategy: Integrate active learning (prompting new users for favorite genres) or hybrid models using metadata embeddings (content-based cold start handles).

6.3 Popularity Bias Analysis

Analyzing recommendations across 100 power users reveals a heavy concentration of blockbusters. The top 5 recommended movies appear in over 40% of all user recommendation lists, indicating a systemic popularity bias that reduces catalog coverage. Applying a frequency penalty or discount on items with high popularity can alleviate this.

7. Dashboard & Explainability

We built an interactive, dark-mode Streamlit dashboard allowing users to:

- Get dynamic personalized movie lists with predicted ratings and rank orders.
- View natural language explanations for recommendations based on their top watch history (e.g. *"We recommend 'Gladiator' because you highly rated 'Braveheart'."*).
- Browse movie popularity counts and search for films.
- Examine performance comparison metrics and live charts.

8. Conclusion & Future Directions

We developed a robust ML recommendation engine. While deep NCF models offer high capacity, regularized SVD Matrix Factorization remains the gold standard for explicit sparse feedback due to its training speed, generalization capabilities, and low prediction latency.

Future work will focus on:

1. **Hybrid Filtering:** Merging text descriptions (using transformer embeddings) with collaborative signals to address the cold start problem.
2. **Session-Based Recommendations:** Using recurrent units (GRUs) or Transformers to predict user actions in sequence.
3. **Unbiased Evaluation:** Incorporating inverse propensity scoring to correct for logging policy bias.